

Area	Evidence	Why it mattered
Frontend design and page structure	0db3caa, 3380ff5, 6b5838c, 4ee2357, 6047275, 0da6dca	I worked heavily on the Landing and Home frontend, including navbar/button design, page layout, visual polish, and the time-of-day background feature.
Login and routing	e3fa573, ab0aff5, 7e4529d, 2ae75f5	I helped build the login/signup structure, page navigation, and the two-case login flow where users with a group go to Home and users without a group go to GroupSetup.
Protected route idea	Emma implemented 60787d6 and a08cf9a	I pushed for a centralized route guard so private pages could not be reached just by typing a URL. This made authentication a real boundary instead of just a form.
Home and group state	0da6dca, d90b736, 0c616ef, ffa23c6, 3deb43d, 03c83d0	I built Home around real backend state, including the grouped dashboard, no-group empty state, join code, roommate count, chores, and expense summary.
Group setup/profile membership	2ae75f5, 582059b, 03c83d0	I worked on create/join group behavior, leave-group behavior, and keeping auth state, page state, and database membership consistent.
Postgres group/user relationship	models.py; RoommateGroupMember; group_routes.py	I helped keep users and groups independent by representing membership with user_id and group_id in a separate table.
Seed data and testing baseline	d90b736, 0c616ef, 519dd7b; backend/seed.py	I created the DEMO123 group and an independent test@example.com user so the team could test both grouped and ungrouped flows repeatedly.
Workflow/orchestration	Tommy/Michelle/Jerry/Kelvin branches, merge history	I coordinated separate branches, frontend file organization, file ownership, pulling origin/main, comparing diffs, and keeping a working version available.

I also contributed heavily to group membership behavior. I worked on the flow where users could create a group, join a group, leave a group, and then have the rest of the app update correctly afterward. This connected the frontend, Flask routes, auth context, and PostgreSQL database together. For example, when a user joined or left a group, the app needed to update both the backend membership data and the frontend user state so Home and Profile would render the correct information.

On the database side, I helped keep our group/user relationship modeled in a clean relational way. Instead of storing a group_id directly inside each user, the project used a separate RoommateGroupMember table to connect user_id and group_id. This was better practice because users and groups could remain independent records, while the relationship between them was stored separately. It also made testing easier because the independent test@example.com user could exist without belonging to the demo group, while the demo users could be linked to DEMO123 through membership rows.

Another contribution I made was creating stable seed data. I created a demo group with the join code DEMO123 and five demo users, while keeping test@example.com outside of that group. This gave the entire team a consistent testing setup. Teammates could test the grouped-user flow with the demo group, or test the new-user/no-group flow with the independent test account. This made bugs much easier to reproduce because we were not all testing with random local data.

Finally, I contributed to the team workflow itself. I pushed the group toward working on separate branches, pulling from origin/main before merging, and keeping features in separate files when possible. When merge issues happened, I used tools like git log --oneline, git diff, and rebasing to